

PROJECT REPORT

STUDENT TO-DO LIST MANAGER

A Modular Python Desktop Application for Academic Task Management

Submitted by:	Prachi Singh
Registration No.:	26BCE11448
Degree & Branch:	B.Tech Computer Science and Engineering (Core)
School:	SCOPE (School of Computing Science and Engineering)
Institution:	VIT Bhopal University
Technology Stack:	Python 3, Tkinter GUI, JSON Storage, Pytest, Git & GitHub
Academic Year:	2026–2027

1. Acknowledgement

I sincerely express my gratitude to my mentor and course faculty for their continuous guidance, valuable feedback, and encouragement throughout the design and development of the Student To-Do List Manager project. I also thank the School of Computing Science and Engineering (SCOPE) and VIT Bhopal University for providing the academic infrastructure, lab facilities, and environment that made this project possible.

2. Introduction

The **Student To-Do List Manager** is a dedicated desktop productivity application developed in Python. It provides a centralized workspace for students to record, organize, prioritize, and track their academic assignments, course projects, lab submissions, and daily personal tasks. By providing immediate visual indicators for task urgency, deadlines, and categories, the application helps students streamline their schedule and avoid last-minute stress.

The project demonstrates core software engineering disciplines, including object-oriented programming with Python dataclasses, graphical interface design with Tkinter, persistent storage using JSON serialization, strict input validation, multi-criteria search and filtering algorithms, dynamic analytics calculations, automated unit testing, and Git version control.

3. Problem Statement

Students continuously handle assignments, practical exams, project milestones, and personal commitments spread across multiple courses. In traditional practice, tasks are either jotted down informally in notebooks, scattered across chat messages, or kept as sticky notes. This unstructured approach causes major issues:

- **Lack of Centralization:** Important deadlines get misplaced or forgotten among informal notes.
- **Poor Urgency Visibility:** Students cannot readily identify which pending tasks are high-priority or already overdue.
- **No Progress Tracking:** Without completion analytics, students lack visibility into their overall academic task completion rate.

4. Objectives

The primary objectives of the application are:

- **Centralized Task Repository:** Enable students to create, view, update, and manage all academic tasks in one place.
- **Persistent Local Storage:** Ensure task data is automatically stored in a local JSON format that remains available across restarts.
- **Robust Data Validation:** Guarantee data integrity by enforcing mandatory titles, valid priority levels, and ISO date standards.
- **Fast Search & Filtering:** Provide real-time searching by title, description, or category, combined with status and priority filters.
- **Productivity Analytics:** Automatically calculate total, completed, pending, overdue, and high-priority task counts along with completion percentage.
- **Responsive User Interface:** Deliver a lightweight, clean graphical desktop user interface using Python Tkinter.

5. System Requirements

Hardware Requirements:

- **Processor:** Dual-Core CPU 1.8 GHz or faster (Intel / AMD / Apple Silicon)
- **RAM:** Minimum 4 GB RAM (8 GB recommended)

- Storage: At least 100 MB free hard drive space
- Display: Minimum 1024 × 768 screen resolution

Software Requirements:

- Operating System: Windows 10/11, macOS, or Linux
- Programming Language: Python 3.10 or higher
- GUI Framework: Tkinter (included with standard Python distribution)
- Testing Framework: Pytest 8.0+
- Development Tools: VS Code, Git, and GitHub

6. Functional Requirements

The system is divided into modular functional units:

- **Task Creation:** Users can create tasks with Title, Description, Due Date (YYYY-MM-DD), Priority (Low, Medium, High), and Category.
- **Task Completion:** Users can select any pending task and mark its status as 'Completed' with a single click.
- **Task Deletion:** Users can delete obsolete tasks with an explicit confirmation dialog.
- **Search & Filtering:** Real-time substring search across title, description, and category, combined with status and priority dropdown filters.
- **Data Persistence:** Automatic serialization of all tasks into a structured todos.json file.
- **Analytics Calculation:** Dynamic computation of total count, completed count, pending count, overdue count, and completion rate percentage.

7. Non-Functional Requirements

- **Usability:** Intuitive desktop form and clean multi-column table layout with minimal learning curve.
- **Reliability:** Rigorous input validation prevents corrupted dates and malformed tasks from reaching storage.
- **Maintainability:** Clean separation of concerns across models, storage, validators, search, analytics, and UI modules.
- **Performance:** Immediate sub-millisecond in-memory filtering and fast JSON read/write operations.
- **Portability:** Runs seamlessly on any operating system supporting Python without requiring external databases or internet connectivity.

8. System Architecture & Design Decisions

The application adopts a **Layered Modular Architecture** separating visual presentation from business logic and data access:

- **Presentation Layer (ui.py):** Handles the Tkinter window, input fields, treeview table, and event dispatching.
- **Service Layer (task_manager.py):** Coordinates CRUD operations and acts as the central business logic controller.
- **Utility Services (validators.py, search.py, analytics.py):** Pure functional modules responsible for input validation, querying/filtering, and statistical metrics.
- **Persistence Layer (storage.py & models.py):** Encapsulates task data classes and disk I/O with JSON formatting.

Design Rationale:

- **Python 3 & Tkinter:** Chosen because Tkinter is bundled with the standard library, requiring zero third-party GUI installations.

- **JSON Persistent Storage:** Eliminates the complexity of database server administration (e.g. MySQL/PostgreSQL) while offering human-readable, portable records.
- **Dataclasses:** Utilizes Python's native `@dataclass` decorator for type clarity, clean serialization (`to_dict`), and structured instantiation.

9. Implementation Details

Module	File	Responsibilities
Data Model	<code>models.py</code>	Defines the <code>Todo</code> dataclass (<code>id</code> , <code>title</code> , <code>description</code> , <code>due_date</code> , <code>priority</code> , <code>category</code> , <code>status</code>).
Task Manager	<code>task_manager.py</code>	Implements <code>TodoManager</code> class: <code>next_id</code> , <code>add</code> , <code>update</code> , <code>delete</code> , <code>complete</code> , and <code>all</code> .
Validation	<code>validators.py</code>	Validates non-empty title, ISO YYYY-MM-DD date format, and valid priority sets.
Storage	<code>storage.py</code>	Loads and writes JSON records to <code>data/todos.json</code> with directory auto-creation.
Search & Filter	<code>search.py</code>	Multi-condition filter checking keyword against title/description/category and status/priority.
Analytics	<code>analytics.py</code>	Calculates total, completed, pending, overdue (using system date), and completion rate %.
User Interface	<code>ui.py</code>	Tkinter desktop GUI with input form, search controls, Treeview table, and stats bar.
Entry Point	<code>main.py</code>	Top-level executable launching the Tkinter event loop via <code>ui.run()</code> .

10. Testing & Verification Matrix

Automated unit testing is implemented using Pytest (`todo_app/tests/test_todo.py`). The following matrix summarizes the test coverage and verified outcomes:

Test ID	Feature Tested	Input / Action	Expected Result	Status
TC01	Title Validation	Empty title ""	Rejected with validation error	PASS
TC02	Date Validation	Invalid date 'wrong-date'	Rejected with format error	PASS
TC03	Valid Task Creation	Valid title, date & priority	Accepted and assigned unique ID	PASS
TC04	Search Query	Keyword 'python'	Only matching tasks returned	PASS
TC05	Status Filter	Filter = 'Completed'	Only completed tasks returned	PASS
TC06	Priority Filter	Filter = 'High'	Only high-priority tasks returned	PASS
TC07	Task Completion	Mark task as Completed	Status toggled to 'Completed'	PASS
TC08	Task Deletion	Delete task by ID	Task removed from list & storage	PASS
TC09	Analytics Metric	Calculate completion rate	Accurate % matching formula	PASS
TC10	Persistence	Restart application	Saved tasks loaded from JSON	PASS

11. Application Screenshots & User Interface Walkthrough

The following screenshots show the live Python Tkinter application in operation, demonstrating core features including task loading, data entry validation, search querying, priority filtering, and dynamic analytics:

Figure 1: Main Application Interface & Initial Dashboard

The screenshot displays the 'Student To-Do List Manager' application window. At the top, there is a 'Create To-Do' section with input fields for 'Task Title', 'Description', and 'Due Date'. Below these are dropdown menus for 'Priority' (set to 'Medium') and 'Category', along with 'Add Task' and 'Complete' buttons. A search bar and filters for 'Status' (set to 'All') and 'Priority' (set to 'All') are located below the form. A 'Delete Selected' button is on the right. The main area contains a table with 8 columns: ID, Title, Description, Due Date, Priority, Category, and Status. The table lists 4 tasks. At the bottom, a live analytics status bar shows: 'Total: 4 | Completed: 1 | Pending: 3 | Overdue: 0 | High Priority: 1 | Completion Rate: 25.0%'.

ID	Title	Description	Due Date	Priority	Category	Status
1	C++	Study	2026-09-25	High	Coding	Completed
2	Data Structures Lab Report	Implement AVL Trees and write analysis secti	2026-09-24	High	Academic	Pending
3	DBMS Quiz Preparation	Review ER modeling, relational algebra, and	2026-09-28	Medium	Study	Pending
4	Hackathon Team Registration	Submit team roster and problem statement p	2026-09-30	Low	Extracurricular	Pending

Total: 4 | Completed: 1 | Pending: 3 | Overdue: 0 | High Priority: 1 | Completion Rate: 25.0%

Figure 1: Shows the main window with all tasks loaded into the multi-column Treeview table and the live analytics status bar at the bottom.

Figure 2: Task Creation Form & Metadata Input

This screenshot shows the same application window as Figure 1, but with the 'Create To-Do' form filled out. The 'Task Title' is 'Operating Systems Assignm', 'Description' is 'CPU Scheduling algorithms sim', and 'Due Date' is '2026-10-02'. The 'Priority' dropdown is set to 'High' and the 'Category' dropdown is set to 'Core CSE'. The 'Add Task' and 'Complete' buttons are visible. The table below the form remains the same as in Figure 1. The live analytics status bar at the bottom shows: 'Total: 4 | Completed: 1 | Pending: 3 | Overdue: 0 | High Priority: 1 | Completion Rate: 25.0%'.

ID	Title	Description	Due Date	Priority	Category	Status
1	C++	Study	2026-09-25	High	Coding	Completed
2	Data Structures Lab Report	Implement AVL Trees and write analysis secti	2026-09-24	High	Academic	Pending
3	DBMS Quiz Preparation	Review ER modeling, relational algebra, and	2026-09-28	Medium	Study	Pending
4	Hackathon Team Registration	Submit team roster and problem statement p	2026-09-30	Low	Extracurricular	Pending

Total: 4 | Completed: 1 | Pending: 3 | Overdue: 0 | High Priority: 1 | Completion Rate: 25.0%

Figure 2: Form populated with Title, Description, ISO Due Date (2026-10-02), Priority ('High'), and Category ('Core CSE').

Figure 3: Real-Time Keyword Search Functionality

Student To-Do List Manager

Create To-Do

Task Title

Description

Due Date

Priority

Medium

Category

Add Task

Complete

Search

Lab

Status

All

Priority

All

Delete Selected

ID	Title	Description	Due Date	Priority	Category	Status
2	Data Structures Lab Report	Implement AVL Trees and write analysis secti	2026-09-24	High	Academic	Pending

Total: 4 | Completed: 1 | Pending: 3 | Overdue: 0 | High Priority: 1 | Completion Rate: 25.0%

Figure 3: Live query matching the substring 'Lab', filtering the table dynamically down to the matching task record.

Figure 4: Multi-Criteria Priority Filtering

Student To-Do List Manager

Create To-Do

Task Title

Description

Due Date

Priority

Medium

Category

Add Task

Complete

Search

Status

All

Priority

High

Delete Selected

ID	Title	Description	Due Date	Priority	Category	Status
1	C++	Study	2026-09-25	High	Coding	Completed
2	Data Structures Lab Report	Implement AVL Trees and write analysis secti	2026-09-24	High	Academic	Pending

Total: 4 | Completed: 1 | Pending: 3 | Overdue: 0 | High Priority: 1 | Completion Rate: 25.0%

Figure 4: Priority dropdown set to 'High', isolating urgent pending and completed items for immediate academic focus.

Figure 5: Task Completion & Dynamic Analytics Update

The screenshot displays the 'Student To-Do List Manager' application window. At the top, there is a 'Create To-Do' section with input fields for 'Task Title', 'Description', and 'Due Date'. Below these are dropdown menus for 'Priority' (set to 'Medium') and 'Category', along with 'Add Task' and 'Complete' buttons. A search bar and filters for 'Status' (set to 'All') and 'Priority' (set to 'All') are located below the form. The main area contains a table with 8 columns: ID, Title, Description, Due Date, Priority, Category, and Status. The table lists 4 tasks. Task #2, 'Data Structures Lab Report', is marked as 'Completed'. At the bottom, a summary bar shows: 'Total: 4 | Completed: 2 | Pending: 2 | Overdue: 0 | High Priority: 0 | Completion Rate: 50.0%'.

ID	Title	Description	Due Date	Priority	Category	Status
1	C++	Study	2026-09-25	High	Coding	Completed
2	Data Structures Lab Report	Implement AVL Trees and write analysis section	2026-09-24	High	Academic	Completed
3	DBMS Quiz Preparation	Review ER modeling, relational algebra, and SQL	2026-09-28	Medium	Study	Pending
4	Hackathon Team Registration	Submit team roster and problem statement	2026-09-30	Low	Extracurricular	Pending

Total: 4 | Completed: 2 | Pending: 2 | Overdue: 0 | High Priority: 0 | Completion Rate: 50.0%

Figure 5: Task #2 marked as 'Completed'. The analytics bar instantly recalculates the completion rate to 50.0%.

12. Challenges Faced & Technical Solutions

• Challenge 1: Safe Persistent Storage Across Sessions

Solution: Implemented JSON serialization via `storage.py` with explicit directory verification (`Path.mkdir(parents=True)`) and exception handling for missing or malformed files.

• Challenge 2: Preventing Invalid Date Formats

Solution: Integrated `datetime.strptime` parsing inside `validators.py` to validate ISO format (YYYY-MM-DD) before saving, raising intuitive user-facing error dialogues in Tkinter.

• Challenge 3: Multi-Criteria Dynamic Filtering

Solution: Designed `filter_todos` to evaluate keyword substrings across title, description, and category simultaneously while respecting composite status and priority filters.

• Challenge 4: Clean Modular Architecture

Solution: Decoupled data models, UI logic, and file I/O into separate modules, enabling full unit test execution without needing to instantiate the graphical interface.

13. Learning Outcomes

Through the design, development, and testing of this project, key competencies were achieved:

- **Object-Oriented Programming:** Practical application of Python classes, dataclasses, serialization, and encapsulation.
- **GUI Engineering:** Building event-driven user interfaces, widgets, grid layouts, and Treeview tables using Tkinter.
- **Software Architecture:** Designing layered modular systems with high cohesion and loose coupling.
- **Quality Assurance:** Writing automated tests with Pytest to verify functional edge cases.
- **Version Control:** Leveraging Git and GitHub for tracking code changes and repository management.

14. Future Enhancements

Potential extensions identified for future versions include:

- **In-place Task Editing:** Direct row double-click editing within the GUI Treeview.
- **Calendar View:** Monthly/weekly visual schedule layout for upcoming assignment deadlines.
- **Desktop Notifications:** System tray alerts for tasks approaching their due date.
- **Export Capabilities:** Exporting task summaries to CSV or formatted PDF reports.
- **Cloud Synchronization:** Multi-device sync backed by SQLite or cloud database storage.

15. Conclusion

The Student To-Do List Manager successfully fulfills the requirements of a modular, robust, and user-friendly desktop application. It provides students with an effective tool to stay organized and monitor their academic progress, while exemplifying good Python software engineering practices from conception to implementation and testing.

16. References

1. Python 3 Documentation — Standard Library Reference, <https://docs.python.org/3/>
2. Tkinter Graphical User Interface Documentation — <https://docs.python.org/3/library/tkinter.html>
3. Pytest Automated Testing Framework — <https://docs.pytest.org/>
4. Git Distributed Version Control System — <https://git-scm.com/doc>
5. VITyarthi Build Your Own Project Guidelines — VIT Bhopal University